ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA **TALLER 3:** **Análisis en PlantUML**	Departamento: Departamento de Ciencias de la Computación Carrera: Ingeniería de Software Asignatura: Análisis y Diseño de Software NRC: 30723 Fecha: 04/05/2026 Integrantes Grupo 3: ● Ayuquina Danny. ● Cañarte Saray. ● Villagómez Doménica.

GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Caso base: Gestionar Paciente

1. Caso base: Gestionar Pacientes

El caso de uso Gestionar Paciente consta de cuatro operaciones principales y los campos de entrada siguientes:

Dato	Tipo sugerido	Descripción	Validación mínima
Cédula	Texto corto	Identificador único del paciente.	Obligatoria, no vacía y no repetida.
Nombre completo	Texto	Nombre del paciente mostrado en el formulario y en la tabla.	Obligatorio; no debe contener solo espacios.
Sexo	Texto / Lista	Sexo del paciente.	Obligatorio; valor de lista predefinida.
Edad	Entero	Edad del paciente.	Obligatoria, numérica y mayor que 0.
Domicilio	Texto	Dirección de residencia del paciente.	Obligatorio; no debe estar vacío.
Teléfono	Texto numérico	Número de contacto del paciente.	Obligatorio; formato de teléfono válido.
Fecha de nacimiento	Fecha	Fecha de nacimiento del paciente.	Obligatoria; no puede ser fecha futura.
Ocupación	Texto	Ocupación laboral del paciente.	Obligatoria; no debe estar vacía.
Estado civil	Texto / Lista	Estado civil del paciente.	Obligatorio; valor de lista predefinida.

Botón / Acción	Caso de uso asociado	Descripción de análisis
Ingresar	Ingresar paciente	Registra un nuevo paciente luego de validar cédula, nombre completo y demás datos requeridos.
Modificar	Modificar paciente	Modifica los datos de un paciente existente identificado por su cédula.
Inactivar	Inactivar paciente	Marca como inactivo al paciente seleccionado, siempre que no tenga citas futuras, registrando el motivo.

Consultar	Consultar paciente	Recupera y presenta el listado de pacientes con búsqueda, filtros y visualización de detalles.

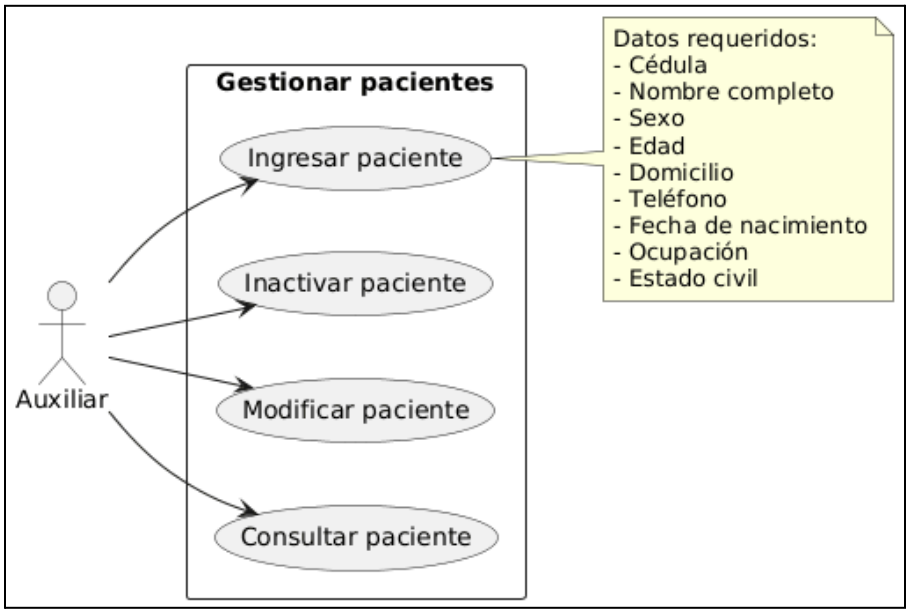
2. Requisitos funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir registrar un paciente con cédula, nombre completo, sexo, edad, domicilio, teléfono, fecha de nacimiento, ocupación y estado civil.	Auxiliar	Ingresar paciente
RF-02	El sistema debe permitir actualizar la información de un paciente existente con validación de datos.	Auxiliar	Modificar paciente
RF-03	El sistema debe permitir inactivar un paciente siempre que no tenga citas futuras programadas, registrando el motivo de inactivación.	Auxiliar	Inactivar paciente
RF-04	El sistema debe permitir consultar pacientes registrados mediante listado, búsqueda y visualización de detalles.	Auxiliar	Consultar paciente

3. Diagrama de casos de uso de análisis

Código PlantUML

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false
actor "Auxiliar" as Aux

rectangle "Gestionar pacientes" {
  usecase "Ingresar paciente" as UC1
  usecase "Inactivar paciente" as UC2
  usecase "Modificar paciente" as UC3
  usecase "Consultar paciente" as UC4
}

Aux --> UC1
Aux --> UC2
Aux --> UC3
Aux --> UC4

note right of UC1
Datos requeridos:
- Cédula
- Nombre completo
- Sexo
- Edad
- Domicilio
- Teléfono
- Fecha de nacimiento
- Ocupación
- Estado civil
end note
@enduml
```



5. Derivación de clases de análisis

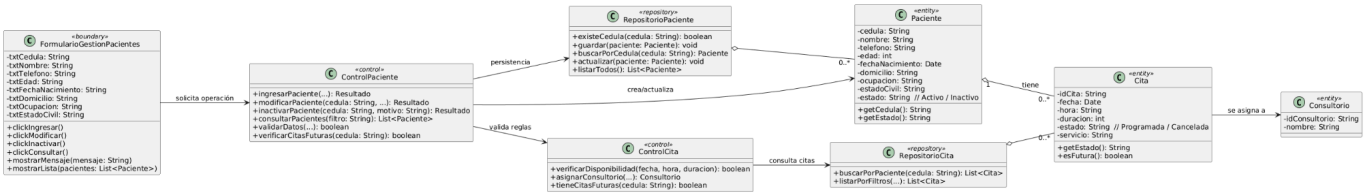
Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el CRUD
Boundary / Frontera	Recibe datos del actor y muestra resultados.	FormularioGestionPacientes	Representa la pantalla con campos Cédula, Nombre, Teléfono, etc. y botones Ingresar, Modificar, Inactivar, Consultar.
Control	Coordina la lógica de cada caso de uso.	ControlPaciente	Valida datos, verifica citas futuras, decide si ingresa, modifica, inactiva o consulta la lista.
Entity / Entidad	Representa la información del dominio.	Paciente	Contiene los atributos cédula, nombre, sexo, edad, domicilio, teléfono, fechaNacimiento, ocupación, estadoCivil, estado.
Entity / Entidad	Representa la información de citas médicas.	Cita	Contiene idCita, fecha, hora, estado. Permite verificar citas futuras al inactivar un paciente.
Repositorio	Conserva y recupera registros de pacientes.	RepositorioPaciente	Permite guardar, buscar por cédula, actualizar, listar y verificar existencia de pacientes.
Repositorio	Conserva y recupera registros de citas.	RepositorioCita	Permite buscar citas por paciente y verificar citas pendientes al intentar inactivar.

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction


class "FormularioGestionPacientes" <<boundary>> {
  - txtCedula: String
  - txtNombre: String
  - txtTelefono: String
  - txtEdad: String
  - txtFechaNacimiento: String
  - txtDomicilio: String
  - txtOcupacion: String
  - txtEstadoCivil: String
  + clickIngresar()
  + clickModificar()
  + clickInactivar()
```

```
  + clickConsultar()
  + mostrarMensaje(mensaje: String)
  + mostrarLista(pacientes: List<Paciente>)
}


class "ControlPaciente" <<control>> {
  + ingresarPaciente(...): Resultado
  + modificarPaciente(cedula: String, ...): Resultado
  + inactivarPaciente(cedula: String, motivo: String): Resultado
  + consultarPacientes(filtro: String): List<Paciente>
  + validarDatos(...): boolean
  + verificarCitasFuturas(cedula: String): boolean
}


class "Paciente" <<entity>> {
  - cedula: String
  - nombre: String
  - telefono: String
  - edad: int
  - fechaNacimiento: Date
  - domicilio: String
  - ocupacion: String
  - estadoCivil: String
  - estado: String  // Activo / Inactivo
  + getCedula(): String
  + getEstado(): String
}


class "Cita" <<entity>> {
  - idCita: String
  - fecha: Date
  - hora: String
  - duracion: int
  - estado: String  // Programada / Cancelada
  - servicio: String
  + getEstado(): String
  + esFutura(): boolean
}


class "Consultorio" <<entity>> {
  - idConsultorio: String
  - nombre: String
}


class "ControlCita" <<control>> {
  + verificarDisponibilidad(fecha, hora, duracion): boolean
  + asignarConsultorio(...): Consultorio
  + tieneCitasFuturas(cedula: String): boolean
}


class "RepositorioPaciente" <<repository>> {
  + existeCedula(cedula: String): boolean
  + guardar(paciente: Paciente): void
  + buscarPorCedula(cedula: String): Paciente
  + actualizar(paciente: Paciente): void
  + listarTodos(): List<Paciente>
}


class "RepositorioCita" <<repository>> {
  + buscarPorPaciente(cedula: String): List<Cita>
  + listarPorFiltros(...): List<Cita>
}
FormularioGestionPacientes --> ControlPaciente : solicita operación
ControlPaciente --> Paciente : crea/actualiza
ControlPaciente --> RepositorioPaciente : persistencia
ControlPaciente --> ControlCita : valida reglas
```

```
ControlCita --> RepositorioCita : consulta citas
Paciente "1" o-- "0..*" Cita : tiene
Cita --> Consultorio : se asigna a
RepositorioPaciente o-- "0..*" Paciente
RepositorioCita o-- "0..*" Cita
@enduml
```



6. Matriz de relación: casos de uso, clases y secuencias

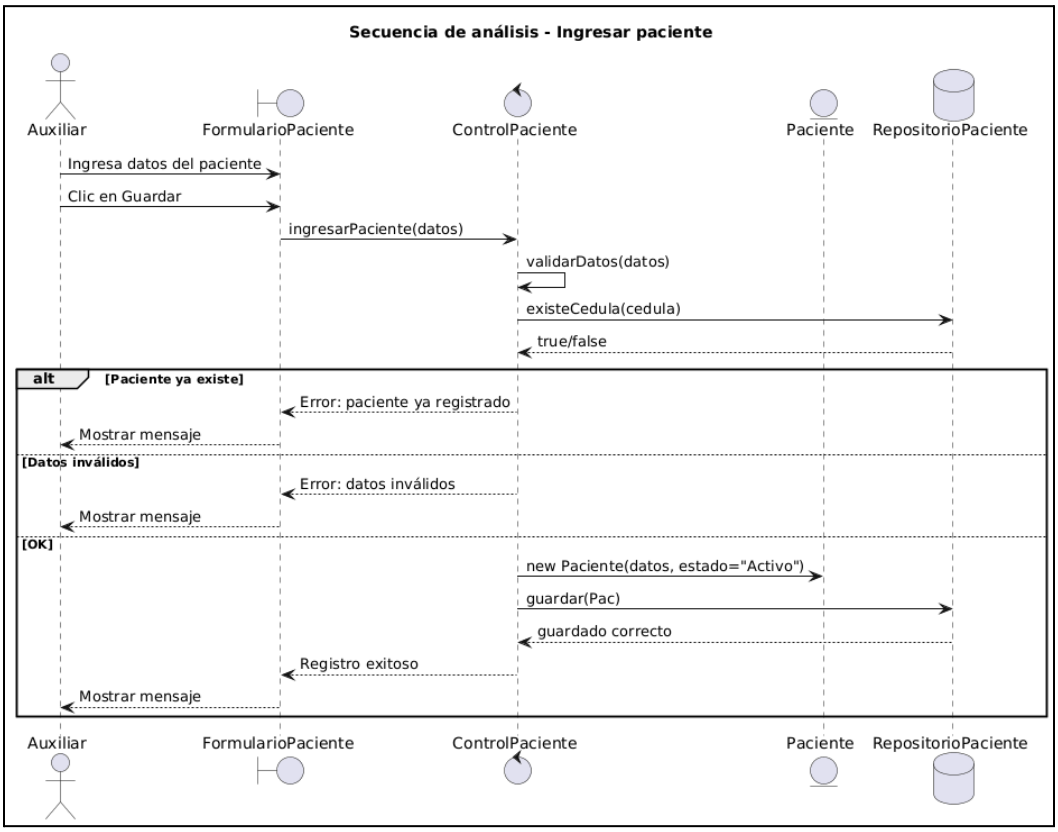
Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Ingresar paciente	Captura cédula, nombre completo y demás datos desde el formulario.	Valida datos, verifica cédula única y coordina el guardado.	Crea Paciente (estado="Activo") y lo guarda en RepositorioPaciente.	Ingresar datos → validar → verificar cédula → crear entidad → guardar → confirmar.
Modificar paciente	Captura cédula e ingresa nuevos valores en el formulario.	Valida datos, busca existencia por cédula y coordina la actualización.	Actualiza Paciente existente en RepositorioPaciente.	Ingresar cambios → validar → buscar → actualizar → confirmar.
Inactivar paciente	Solicita cédula e ingresa motivo de inactivación.	Verifica existencia y consulta citas futuras en RepositorioCita antes de inactivar.	Cambia estado de Paciente a "Inactivo" en RepositorioPaciente.	Ingresar cédula → buscar → verificar citas → ingresar motivo → inactivar → notificar.
Consultar paciente	Solicita mostrar lista, filtra y presenta tabla con detalles.	Solicita todos los registros con filtros opcionales.	RepositorioPaciente devuelve lista de Paciente; permite ver detalle individual.	Solicitar listado → recuperar lista → mostrar tabla → seleccionar → ver detalle.

7. Diagramas de secuencia de análisis

7.1 Secuencia: Ingresar Paciente

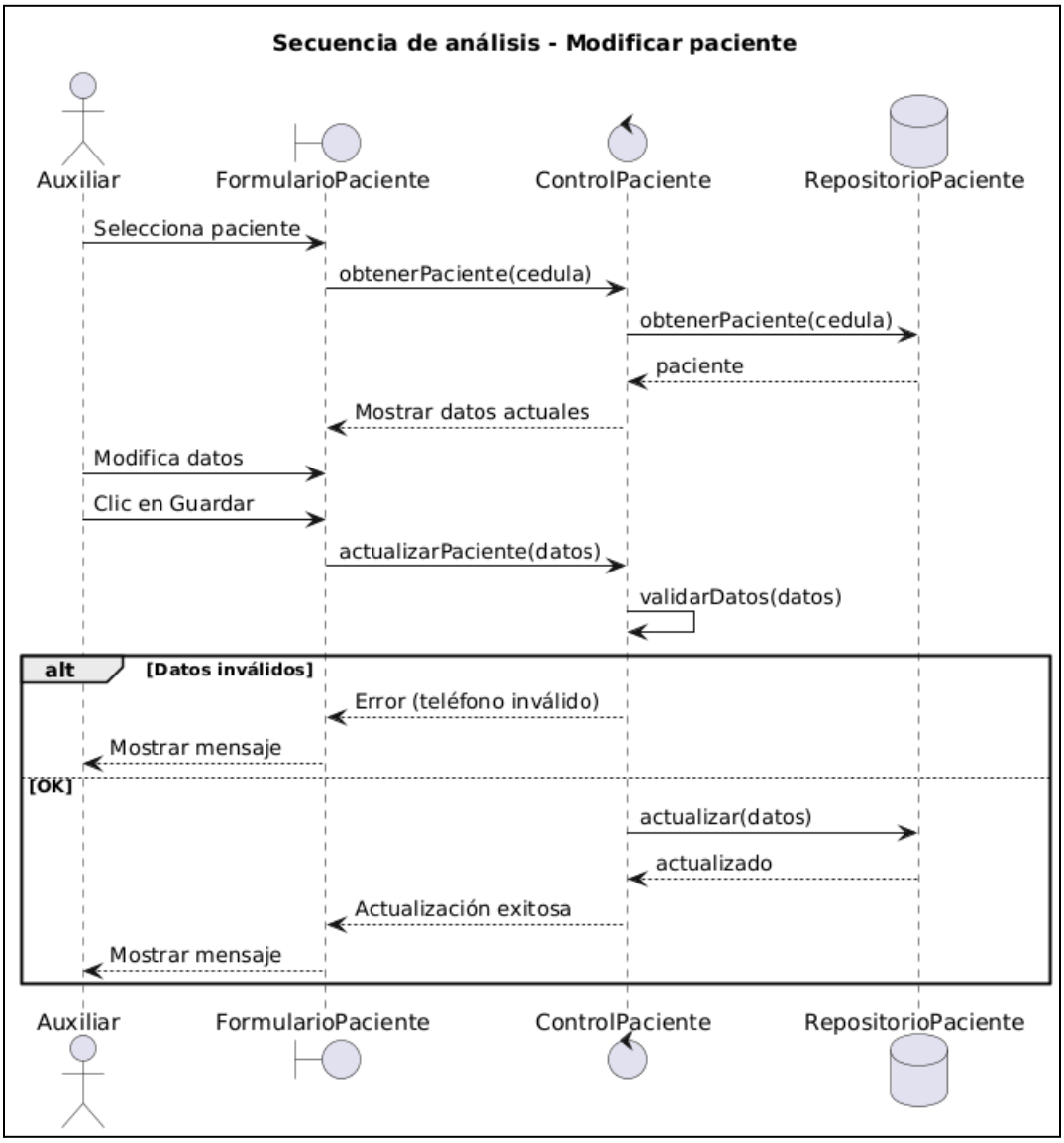
```
@startuml
title Secuencia de análisis - Ingresar paciente
actor "Auxiliar" as Aux
boundary "FormularioPaciente" as UI
control "ControlPaciente" as Ctrl
entity "Paciente" as Pac
database "RepositorioPaciente" as Repo

Aux -> UI : Ingresa datos del paciente
Aux -> UI : Clic en Guardar
UI -> Ctrl : ingresarPaciente(datos)
Ctrl -> Ctrl : validarDatos(datos)
Ctrl -> Repo : existeCedula(cedula)
Repo --> Ctrl : true/false

alt Paciente ya existe
  Ctrl --> UI : Error: paciente ya registrado
  UI --> Aux : Mostrar mensaje
else Datos inválidos
  Ctrl --> UI : Error: datos inválidos
  UI --> Aux : Mostrar mensaje
else OK
  Ctrl -> Pac : new Paciente(datos, estado="Activo")
  Ctrl -> Repo : guardar(Pac)
```

```
  Repo --> Ctrl : guardado correcto
  Ctrl --> UI : Registro exitoso
  UI --> Aux : Mostrar mensaje
end
@enduml
```



8.2 Secuencia: Modificar Paciente

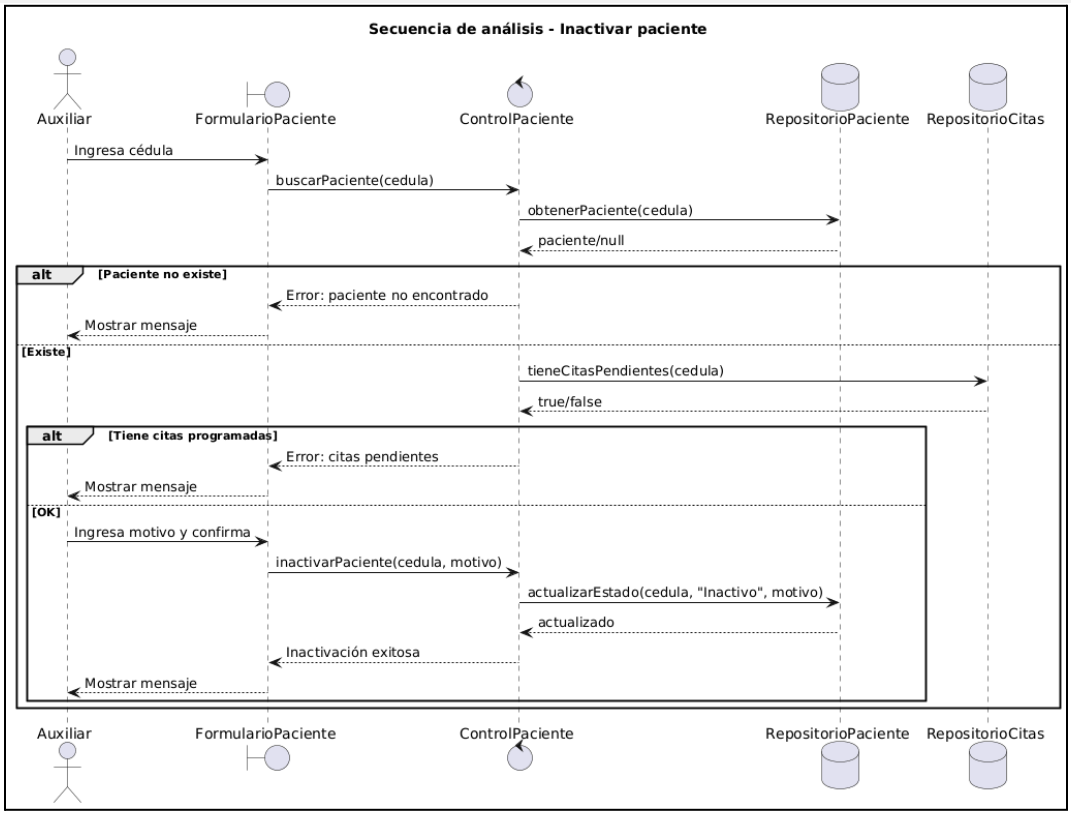
```
@startuml
title Secuencia de análisis - Modificar paciente
actor "Auxiliar" as Aux
boundary "FormularioPaciente" as UI
control "ControlPaciente" as Ctrl
database "RepositorioPaciente" as Repo

Aux -> UI : Selecciona paciente
UI -> Ctrl : obtenerPaciente(cedula)
Ctrl -> Repo : obtenerPaciente(cedula)
Repo --> Ctrl : paciente
Ctrl --> UI : Mostrar datos actuales
Aux -> UI : Modifica datos
Aux -> UI : Clic en Guardar
UI -> Ctrl : actualizarPaciente(datos)
Ctrl -> Ctrl : validarDatos(datos)

alt Datos inválidos
  Ctrl --> UI : Error (teléfono inválido)
  UI --> Aux : Mostrar mensaje
else OK
  Ctrl -> Repo : actualizar(datos)
  Repo --> Ctrl : actualizado
  Ctrl --> UI : Actualización exitosa
  UI --> Aux : Mostrar mensaje
end
@enduml
```



8.3 Secuencia: Inactivar Paciente

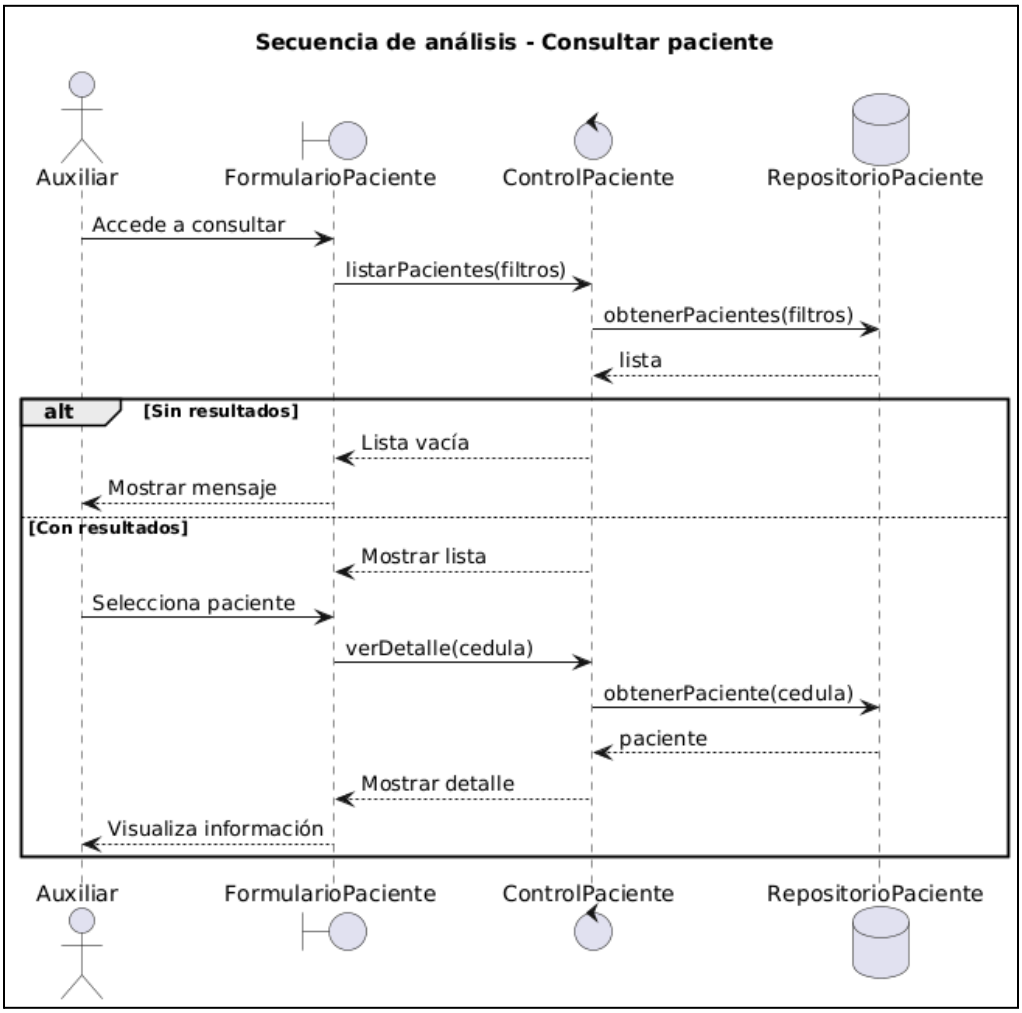
```
@startuml
title Secuencia de análisis - Inactivar paciente
actor "Auxiliar" as Aux
boundary "FormularioPaciente" as UI
control "ControlPaciente" as Ctrl
database "RepositorioPaciente" as Repo
database "RepositorioCitas" as RepoCitas

Aux -> UI : Ingresa cédula
UI -> Ctrl : buscarPaciente(cedula)
Ctrl -> Repo : obtenerPaciente(cedula)
Repo --> Ctrl : paciente/null

alt Paciente no existe
  Ctrl --> UI : Error: paciente no encontrado
  UI --> Aux : Mostrar mensaje
else Existe
  Ctrl -> RepoCitas : tieneCitasPendientes(cedula)
  RepoCitas --> Ctrl : true/false
  alt Tiene citas programadas
    Ctrl --> UI : Error: citas pendientes
    UI --> Aux : Mostrar mensaje
  else OK
    Aux -> UI : Ingresa motivo y confirma
```

```
    UI -> Ctrl : inactivarPaciente(cedula, motivo)
    Ctrl -> Repo : actualizarEstado(cedula, "Inactivo", motivo)
    Repo --> Ctrl : actualizado
    Ctrl --> UI : Inactivación exitosa
    UI --> Aux : Mostrar mensaje
  end
end
@enduml
```



8.4 Secuencia: Consultar Paciente

```
@startuml
title Secuencia de análisis - Consultar paciente
actor "Auxiliar" as Aux
boundary "FormularioPaciente" as UI
control "ControlPaciente" as Ctrl
database "RepositorioPaciente" as Repo

Aux -> UI : Accede a consultar
UI -> Ctrl : listarPacientes(filtros)
Ctrl -> Repo : obtenerPacientes(filtros)
Repo --> Ctrl : lista

alt Sin resultados
  Ctrl --> UI : Lista vacía
  UI --> Aux : Mostrar mensaje
else Con resultados
  Ctrl --> UI : Mostrar lista
  Aux -> UI : Selecciona paciente
  UI -> Ctrl : verDetalle(cedula)
  Ctrl -> Repo : obtenerPaciente(cedula)
  Repo --> Ctrl : paciente
  Ctrl --> UI : Mostrar detalle
  UI --> Aux : Visualiza información
end
@enduml
```



8. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Ingresar paciente	La cédula no debe existir previamente en el sistema.	Ingresar cédula, nombre completo y demás campos → validar datos → verificar cédula única → guardar paciente.	Paciente registrado correctamente con estado "Activo" y visible en la lista.
Modificar paciente	El paciente debe existir y estar registrado en el sistema.	Seleccionar paciente por cédula → mostrar datos actuales → modificar campos → validar → guardar cambios.	Datos del paciente actualizados correctamente.
Inactivar paciente	El paciente debe existir y no tener citas futuras programadas.	Ingresar cédula → buscar paciente → verificar citas futuras → ingresar motivo → confirmar inactivación.	Paciente marcado como "Inactivo" con motivo registrado; no aparece en listado activo.
Consultar paciente	Pueden existir cero o más pacientes registrados.	Acceder a consultar → aplicar filtros opcionales → recuperar registros → mostrar tabla → seleccionar → ver detalle.	Tabla actualizada con listado de pacientes y detalle de información al seleccionar uno.

9. Conclusión técnica

A lo largo de este documento se evidencia que el análisis no se construye como un conjunto de artefactos aislados, sino como una red de elementos independientes cuya coherencia determina la calidad del diseño

posterior. En el caso de gestión de pacientes, cada nivel de abstracción (casos de uso, clases de análisis y diagramas de secuencia) cumple una función específica y relevante dentro del proceso.

Los requisitos abarcados establecen el funcionamiento entre el actor (auxiliar) y el sistema, definiendo las operaciones disponibles de ingresar, modificar, inactivar y consultar paciente. Sin embargo, los casos de uso por sí solos no explican cómo el sistema cumple esas responsabilidades internamente, de modo que, el diagrama de clases de análisis distribuye estas para delimitar los roles y los diagramas de secuencia integran ambas perspectivas para mostrar la relación entre elementos.

En este sentido, un modelo con esta estructura de análisis facilita la transición hacia el diseño e implementación, las clases de análisis evolucionan directamente hacia clases de diseño con responsabilidades ya definidas, los métodos identificados constituyen la interfaz y las relaciones del diagrama de clases orientan las decisiones sobre dependencias y datos.

10. Fuente interna de referencia

- 01_Guia_PlantUML_CU_Analisis_CRUD_Estudiante.
- Guía Version2 del CRUD Estudiante con campos ID, Nombre y Edad, adaptada a la interfaz de referencia proporcionada por la docente.